Case Study: InfiniteWeb

Generated websites as training worlds for GUI agents

Zhang, Wang, Zhang, Guo, Li, Li, Lu · Peking Univ., Nanjing Univ., Microsoft Research Asia · 2026

Learning outcomes

By the end of this session, you can:

1. Explain how InfiniteWeb turns one Common Crawl page into a working website.
2. Explain “task-centric correctness” and the TCTDD loop.
3. Read a generated evaluator and explain why partial credit helps GRPO.
4. Compare InfiniteWeb with Agent World Model.
5. Read the transfer results to desktop and mobile with care.
6. Plan an InfiniteWeb-style environment for Indian websites.

The problem: GUI agents have few places to practise

Benchmarks

WebArena, OSWorld

Tens to hundreds of apps

OSWorld: ~1,800 person-hours

Synthetic trajectories

OS-Genesis, AgentTrek

New data, same old apps

Risk: overfitting

LLM-written sites

One page: fine

Many pages: inconsistent

Bugs → wrong rewards

Three challenges: consistency, correctness, diversity.

Derive everything from tasks. Test only what the tasks need.

Consistency → one shared spec · correctness → task-centric tests · diversity → real seeds and designs

InfiniteWeb at a glance

$1.93
per website · ~20 min median

8–10
tasks per site · ≤ 12 pages

+6.9
OSWorld points · 24.5 → 31.4

Builder GPT-5, reasoning “high”, temperature 0.7

Learner UI-TARS-1.5-7B, GRPO, screenshots in, clicks out

Training set 600 tasks, in steps of 200 / 400 / 600

InfiniteWeb vs Agent World Model

	AWM	InfiniteWeb
Agent sees	tool results (JSON)	screenshots
Agent acts	MCP tool calls	click, type, scroll
State	SQLite database	browser localStorage
Reward	code + GPT-5 judge	code only, partial credit
Learner	Qwen3 (4B–14B)	UI-TARS-1.5-7B
Parallel envs	1,024	128

Same recipe: LLM writes the world, code checks it, GRPO trains on it.

The website as a POMDP

Part	In InfiniteWeb
State s	localStorage + current page and URL
Observation o	screenshot only, no storage view
Action a	click(x, y), type(text), scroll, finish
Transition T	the site’s own JavaScript
Reward r	evaluator score, 0 to 1, at the end
Horizon	at most 15 steps

“Partially observable”: cart contents off-screen, order records hidden.

Part 1

How they generate at scale

From one Common Crawl page to a working, multi-page website

The four-stage pipeline (Figure 3)

1 · Unified Specification

tasks, data models, pages, interfaces

2 · Task-centric backend

data + business logic, fixed by tests

3 · Design-guided frontend

HTML, CSS, UI JS from a design image

4 · Evaluator generation

one code checker per task

Stages 2 and 3 run in parallel, both read the same spec

Two inputs per site, both from Common Crawl

Website seed

what the site does

e.g. “wedding planning”, “forklift dealer”

LLM writes it from the page

Design image

how the site looks

headless render → screenshot

often from a different page

Filter drop robots.txt violations and illegal content

Why it scales billions of crawled pages → seed × design pairs

Stage 1a: tasks come first (Figure 14)

Rules concrete values · clear decision criteria · no email or external service

	Example from the prompt
BAD	“Compare products and select the best one”
GOOD	“Compare two laptops and select the one with more RAM under $1000”
BAD	“Search for headphones and add to cart”
GOOD	“… wireless headphones under $200, 4+ stars, add the first result”

Steps actions only: “Click the search button in the header”, no “Verify …”

Stage 1b: data models, pages, interfaces

Data models Product, Cart, Order · size “many” 10–20, “few” 3–5

Pages home, search results, product details, cart, checkout

Interfaces one signature, shared by every page

```
addToCart(userId, sessionId, productId, quantity)
  → addToCart(productId, quantity)
```

Wrapping rule “Would a user type this value?” No → hide it in state

Hidden too cartId, authToken, locale, currency

Stage 2: backend with task-centric TDD

Key idea agents walk only task paths → only those must work

Data first task “under $50” → catalog must hold items under $50

Images placeholders → real photos via Unsplash, Picsum

Tests + code written in parallel, from the same data

Example test data has $29.99 and $45.00 → expect exactly these two

On failure LLM gets failing test + expected vs actual + code

Output BusinessLogic JS class on localStorage, runs in Node and browser

Loop limit at most 8 rounds; most sites need 1–3

TCTDD example: a forklift dealer (Table 4)

```
Iteration 1
  ✓ Task 1: quote for three forklifts
  ✓ Task 2: two electric forklifts to cart
  × Task 3: schedule demo for product
  9/10 passed
Iteration 2  10/10 passed
```

Bug submitDemoRequest() returned undefined, not a success object

Across all sites 1.5% still failing after 8 rounds

Stage 3: design-guided frontend

Style extraction VLM reads the design image

colour palette · fonts and weights · spacing · button patterns

Page design content blocks, routing by URL parameter, grid breakpoints

e.g. product.html?id=p12

Shared frame one header, footer, CSS variables for all pages

Per page HTML + CSS + thin UI JavaScript

```
<button data-action="addToCart">
```

Init script homepage writes the generated data into localStorage

What one website costs (Appendix F)

Item	Value
Tokens per site	0.36M in + 0.34M out
Money per site	≈ $1.93
Time per site	≈ 20 minutes (median)
600 sites, in parallel	≈ 125 minutes
Share of training time	6.9%
OSWorld, by hand	~1,800 person-hours, 369 tasks

Rough sum 600 sites ≈ $1,160 in API cost

Part 2

How they verify the websites

Inside the pipeline: tests. Outside it: benchmarks, judges and people.

Four checks on quality

Check	Question	Where
TCTDD tests	Do task paths work?	every site, §3.3
WebGen-Bench	Can an agent finish tasks?	101 sites, §4.2
GPT-5 judge	Does it match the design?	200 sites, §4.3
Humans	Are tasks and judges right?	100 + 100, App. D–E

Only TCTDD runs in production; the rest are measurements

Compare AWM AWM checked by execution; same spirit

Function: WebGen-Bench (Table 1)

Setup same seed + design image · all use GPT-5, “high”

Builder	Functional	Design valid.	Overall
Bolt.diy	58.4	41.7	67.0
Claude-Code	67.3	61.1	74.3
Codex	72.8	76.4	81.2
InfiniteWeb	**80.9**	**82.8**	**85.6**

Scoring agent tries each task → Passed / Partial / Failed

Looks: GPT-5 as visual judge (Figure 6)

Input design image + two full-page screenshots

Five criteria layout · colours · type and spacing · components · overall feel

Verdict ours wins / baseline wins / tie

InfiniteWeb vs	Win rate (Table 8)
Claude-Code	85.4%
Bolt.diy	84.2%
Codex	69.3%

Judge check 100 cases, humans agree 91%

People check the tasks (Appendix E)

95 / 100
sampled tasks pass human review

1.5%
sites still failing after 8 TCTDD rounds

Question 1 task clear and doable on this site?

Question 2 evaluator right about done vs not done?

Class point 5% noise reaches the reward; GRPO must tolerate it

Ablation: what each part adds (Figure 7)

Variant	WebGen-Bench overall
Full InfiniteWeb, GPT-5	**85.6**
Without TCTDD loop	82.6 (Table 6)
GPT-4.1 as builder	77.4
Claude-Code, GPT-5	75.8 (Table 5)

Reading 1 test loop helps, even with a strong model

Reading 2 pipeline + weaker model > agent + stronger model

Caution text says −5.0 (80.6); Table 6 says 82.6

Part 3

How they score the agent

Code evaluators with weighted checkpoints: a dense reward, no judge

Stage 4: one evaluator per task

Existing variables state the site keeps anyway: cart, orders, preferences

Instrumentation variables new flags for task progress

name: task1_searchCompleted

set inside searchNeighborhoods(), before return

wrapped in try-catch · site behaviour unchanged

Example flags query sent → product viewed → added to cart → paid

Evaluator JS function, reads localStorage, returns 0.0 to 1.0

A real evaluator, simplified (Figure 12)

Task restaurant site · “Subscribe to newsletter with weekly specials”

```
a = get('task10_subscriptionAttempt')
c = get('task10_subscriptionConfirmed…')
subs = get('newslettersubscriptions')
cp1 = a.email ok && a.wantsWeekly // 0.35
cp2 = subs has a.id, same email // 0.30
cp3 = c.id == a.id                // 0.35
return sum of passed weights
```

Partial run attempt only → 0.35 · full flow → 1.0

Why partial credit helps GRPO (Figure 8)

GRPO advantage reward – group mean

Problem all 0 or all 1 in a group → advantage 0 → no gradient

Test 4,000 tasks × 4 rollouts of UI-TARS-1.5-7B

174

useful tasks, 0/1 reward

767

useful tasks, dense reward · 4.4×

Example group 0, 0, 0, 0 → nothing · 0, 0.35, 0.35, 0.65 → signal

Reward hacking and its limits

Shortcut write the order straight into localStorage

Block flags set only inside business functions → shortcut fails

Also agent acts by pixels only, no DevTools, no JS console

Open risks (our reading, not tested in the paper)

- evaluator checks flags, not what the screen showed
- high weight on “attempt” can reward half-done work
- LLM writes both site and evaluator → shared blind spots

Part 4

The RL algorithm

GRPO on a 7B screenshot agent, 128 browsers at once

Training setup (Section 4.4, Appendix B)

Policy UI-TARS-1.5-7B

Algorithm GRPO, clip 0.2–0.3

Optimiser AdamW, lr 1e-6, bf16

Batch global 16, 1 PPO epoch

Other grad clip 1.0, γ = 0.95

Browsers 128 in parallel

Group size 8 rollouts per task

Horizon max 15 steps

Sampling temperature 1.0

Data 200 / 400 / 600 tasks

One step screenshot → UI-TARS thinks → click(x, y) → site JS runs

End evaluator score = reward for the whole trajectory

Part 5

Results and limits

Train on fake websites, test on real web, desktop and mobile

Main result: more tasks, better scores

Benchmark	Base	+200	+400	+600
Online-Mind2Web	23.0	25.3	27.3	**28.7**
OSWorld-Verified	24.5	27.3	29.7	**31.4**
MobileWorld	6.4	–	–	**10.3**

Mind2Web live real sites · in-domain · easy 46.9 → 56.8, hard 5.3 → 9.2

OSWorld desktop apps · out-of-domain · 15-step limit

MobileWorld phone apps · no extra tuning

Trend no plateau yet at 600 tasks

OSWorld by app (Table 2)

App	Base	+600 tasks	Gain
Chrome	22.9	36.9	+14.0
GIMP	51.9	69.2	+17.3
Thunderbird	26.7	40.0	+13.3
OS utilities	34.8	45.8	+11.0
Writer	39.1	47.8	+8.7
Multi-Apps	3.8	9.7	+5.9
Calc · Impress · VS Code	small		+0.1 to +2.2

Note 200 tasks hurt some apps first: VS Code 63.0 → 55.1

Quality, not quantity (Table 9)

Test same 600 tasks, same GRPO · only the site builder changes

Sites built by	Mind2Web	OSWorld	Mobile
No training	23.0	24.5	6.4
Claude-Code	23.7	25.2	5.1
InfiniteWeb	**28.7**	**31.4**	**10.3**

Why Claude-Code keeps the whole codebase in context

→ cross-page bugs · 4.9 vs 12.6 pages per site

Are the generated tasks hard? (Table 3)

Env	Agent	Score	Steps
InfiniteWeb	UI-TARS-1.5-7B	7.4	10.3
InfiniteWeb	Agent S2	14.1	10.9
OSWorld	UI-TARS-1.5-7B	24.5	9.0
OSWorld	Agent S2	27.3	9.3

Harder 2–3× lower scores, longer successful runs

Sharper gap between agents: 6.7 points vs 2.8 on OSWorld

Agent S2 GPT-4.1 planner + UI-TARS-72B grounding

What transfers? Three cases (Appendix A.1)

Persistence

VS Code: set Arabic

base: “not visible”, quits

trained: types “Arabic”

Full flow

Install Spotify

base: stops at search, 4 steps

trained: installs, 13 steps

No loops

Thunderbird attachment

base: open/cancel ×11

trained: done in 5

Claim habits transfer, not app knowledge

Limits

The paper says

One site per task no “compare prices across shops”

Web only no native mobile apps

Cost ~20 min and $1.93 per site

We add

Single user no login, OTP, roles, other users

Calm world no pop-ups, CAPTCHAs, slow pages, errors

Scale shown 600 tasks, not “infinite”

Part 6

InfiniteWeb for India

Booking, payment, document and scheme websites, in Indian languages

What to borrow, what to change

Paper idea	Indian version
Common Crawl seeds	.in and Indic-language pages
Tasks with concrete values	₹, lakh, PNR, IFSC, dates DD/MM
TCTDD on task paths	+ tests for Indian rules (GST, quota)
Hide locale, currency	make them task variables
Weighted checkpoints	keep as is
Single user, no login	add OTP, KYC, second user
One site per task	add cross-site tasks

Six site families to generate

Family	Example task
Rail booking	Tatkal sleeper, lower berth for senior
UPI-style payment	Pay ₹1,250 to a VPA, add note
Document wallet	Fetch marksheet, share with college
Scheme portal	Check eligibility, apply, track status
Net banking	Add payee by IFSC, then NEFT
Kirana / ONDC-style	Order in Hindi, GST invoice

Names fictional brands only, e.g. “RailYatra”, not IRCTC

Worked example: “RailYatra”

Task 2 sleeper seats, Pune → Nagpur, 12/12, lower berth for the 64-year-old, pay by UPI

Models Train, Station, Availability, Passenger, Booking

Interfaces searchTrains(from, to, date, cls) · book(train, pax) · payUPI(vpa)

TCTDD data 2 trains on the route, 1 with sleeper seats left

Checkpoint	Weight
Search: right route, date, class	0.20
Train with seats chosen	0.20
Senior: age 64 + lower berth	0.30
UPI paid, PNR on screen	0.30

Indic language on screen

UI language Hindi, Tamil, Bengali, Marathi · plus English toggle

Task language same or different from the UI

e.g. Hinglish task on a Tamil-UI site

Formats ₹1,25,000 · 12/12/2026 · +91 numbers · PIN codes

Design images from Indian pages: dense menus, banners, mixed scripts

New test does the text render? no broken conjuncts or boxes

Baseline first measure UI-TARS on Devanagari screens before RL

Ground rules and new task types

Safety

Synthetic people fake names, IDs that fail checksums

Offline only never point the agent at a live portal

Fictional brands no real logos, names or domains

New tasks

OTP step OTP shown in a fake SMS panel

Cross-site get PAN from wallet → fill it in bank KYC

Rule traps payee added < 24 h ago → NEFT limit applies

A class or lab project in four steps

Step	Output
1 · Pilot	20 sites, 2 families, 1 language
2 · Check	50 tasks by hand, as in Appendix E
3 · Scale	200–600 sites, all six families
4 · Train	GRPO on a 7B GUI model

Test on held-out Indian sites + Online-Mind2Web

Site budget 600 × $1.93 ≈ $1,160 at the paper’s rate

Student task one family each: spec, tests, evaluator

Questions for discussion

1. Why is “task-centric correctness” enough for RL but not for a product?
2. The same LLM writes site, tests and evaluator. What can go wrong?
3. Pick checkpoint weights for a 4-step task. How do you stop farming?
4. Why do web skills help GIMP but not Calc?
5. Claude-Code sites gave +0.7. What does that say about scaling data?
6. How would you build a cross-site task with localStorage only?

What to read in class (arXiv v3)

Topic	Read	Page
Big picture	§1, Figure 3	1–3
Generation	§3.2–3.4, Figs 14, 18	3–5, 17, 20
Verification	Table 4, §4.2–4.3, App. E	7–8, 13, 15
Reward	§3.5, Fig 12, Fig 8	6, 8, 14
RL setup	§4.4, App. B	8, 13
Results	Tables 2, 3, 9, App. A.1	6, 8, 11–12, 15
Limits	Limitations	9

Skip related work, most prompts (Figs 15–29)

Read more

InfiniteWeb paper arxiv.org/abs/2601.04126

Code github.com/microsoft/FIVE-UI-Evol

WebGen-Bench arxiv.org/abs/2505.03733

OSWorld arxiv.org/abs/2404.07972

Online-Mind2Web arxiv.org/abs/2504.01382

UI-TARS arxiv.org/abs/2501.12326

MobileWorld arxiv.org/abs/2512.19432

GRPO (DeepSeekMath) arxiv.org/abs/2402.03300